IT404041 REAL TIME PROGRAMMING · ASSIGNMENT 2 · SMART GREENHOUSE CONTROLLER

Contributions

Who codes what – work split for the 5 roles

Generated 2026-11-05

This document splits the whole project (backend + Postman + scripts + report) between the 5 roles of the assignment. Each file has **exactly one owner**. The owner writes it, explains it in the report and presents it.

How you can see the owner:

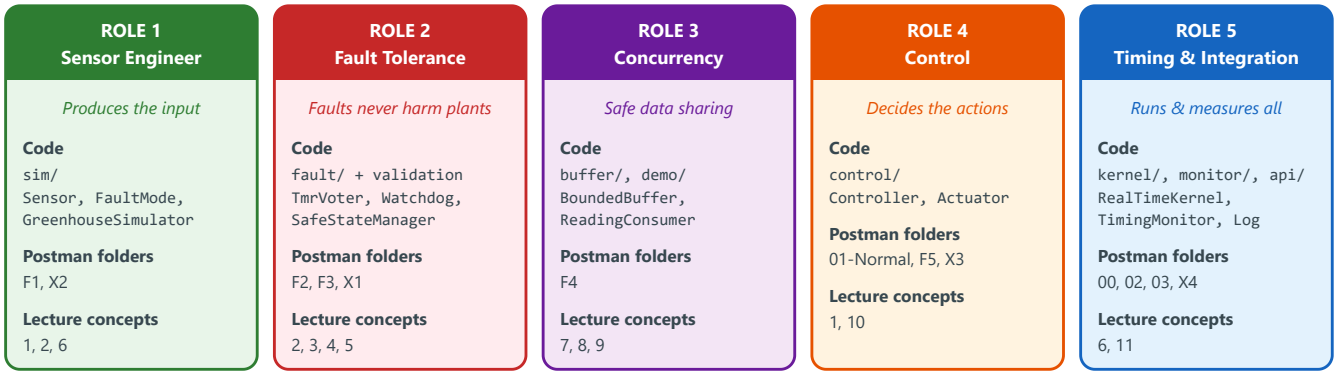
- **Java and script files:** the first comment says `Author: Role N - <role name>`.
- **Postman folders:** the folder description starts with `Owner: Role N - <role name>`.
- **Concept lines:** inside the code, the lines that show a lecture concept are tagged `// CONCEPT n:`. Search for e.g. `CONCEPT 9` to find them.

Fill in the member names:

Role	Member
Role 1 – Sensor Engineer	_____
Role 2 – Fault Tolerance Engineer	_____
Role 3 – Concurrency Engineer	_____
Role 4 – Control Engineer	_____
Role 5 – Timing & Integration Lead	_____

1. Overview – everything on one page

The 5 roles at a glance



Role	Backend (Java)	Postman folders	Other files	Scenarios presented	Lecture weeks
1 Sensor	`sim/` – Sensor, SensorType, FaultMode, Reading, GreenhouseSimulator	F1-WrongValue, X2-StuckAndImpossibleValue	–	F1	1, 2, 6

Role	Backend (Java)	Postman folders	Other files	Scenarios presented	Lecture weeks
2 Fault Tolerance	`fault/` – TmrVoter, VoteStatus, VoteResult, Watchdog, SafeStateManager, ReadingValidator `api/` – GlobalExceptionHandler, dto/EnvironmentRequest, dto/FaultRequest, dto/HeatWaveRequest `kernel/` – UnknownSensorException	F2-DeadSensor, F3-SafeState, X1-InvalidInputs	–	F2, F3	3, 4, 5
3 Concurrency	`buffer/` – BoundedBuffer, ReadingConsumer, LatestReadings `demo/` – RaceConditionDemo	F4-RaceCondition	–	F4	7, 8, 9, 11
4 Control	`control/` – Controller, Actuator, Actuators	01-Normal-Operation, F5-HeatWave, X3-HumiditySoilFailure	–	F5	1, 13
5 Timing & Integration	GreenhouseApplication `kernel/` – RealTimeKernel `monitor/` – TimingMonitor, StatusDisplay, FaultMenu, Log, EventRecorder, InputDescriber `api/` – GreenhouseRestController, dto/ScenarioRequest	00-Common-Setup, 02-Repair-All, 03-Timing-And-Shutdown, X4-MixedChaos, both environments, `build-collection.js`	`pom.xml`, `mvnw*`, `.mvn/`, `application.properties`, `run.bat`, `run.sh`, `run-demo.bat`, `test-scenario.bat`, `test-all.bat`, `README.md`, `CONTRIBUTIONS.md`, `.gitignore`	Timing table, full run	6, 14

All Java files are under `src/main/java/com/greenhouse/` and all Postman files are under `postman/`.

Size of the Java part:

Role	Approx. Java lines
Role 1	270
Role 2	530
Role 3	280
Role 4	320
Role 5	1060

Role 5's share is larger because it also wires everything together (start, stop, logging, REST). That is the integration-lead part of the role.

---

2. Role 1 – Sensor Engineer

> Assignment: Sensor class (Runnable) for temperature, humidity and soil moisture, run every 500 ms. Fault modes wrong value, stuck value, dead (F1, F2). Simple greenhouse simulation.

Backend files

File	What to code / explain	Key parts
`sim/Sensor.java`	One sensor = one periodic task (Runnable) released every 500 ms. Reads the greenhouse, applies its fault mode and puts the reading into the buffer (producer).	`run()` (periodic body, timed for the timing table), `sampleOnce()` (WRONG_VALUE adds the offset, STUCK returns the frozen value, DEAD sends nothing), `injectFault()`, `repair()`, `volatile FaultConfig fault`
`sim/FaultMode.java`	The 4 modes: `NONE`, `WRONG_VALUE`, `STUCK`, `DEAD`	–
`sim/SensorType.java`	Temperature, humidity, soil (with unit)	–
`sim/Reading.java`	Immutable record passed from sensor thread to consumer thread	`record Reading(sensorId, type, value, timestampMs)`
`sim/GreenhouseSimulator.java`	The "physical world": the sun warms (+0.01 °C/s), the heater warms (+0.5), the fan cools (−0.5) and dries the air, the pump wets the soil (+2 %/s), the soil dries (−0.1 %/s), plus a heat wave	`step()`, `read()` (true value + small noise), `setEnvironment()`, `addHeatWave()`, `reset()` – all `synchronized`

Postman folders

Folder	What it shows
F1-WrongValue	T2 +15 °C → T2 reports about 40 °C, but the system still uses the real 25 °C (MASKED), so there is no false alarm
X2-StuckAndImpossibleValue	T1 STUCK while the temperature rises → out-voted; T2 +90 → 115 °C reading → rejected

Report section

Sensors, periodic tasks (RT1, 500 ms, firm), and the fault → error → failure table:

Fault (cause)	Error (wrong state)	Failure if not handled	Prevented by
T2 gives +15 °C	Reading 40 °C instead of 25 °C	Fan + alarm switched ON wrongly	TMR voter masks T2 (MASKED)
T3 dead	No readings from T3	Controller uses old data	Watchdog marks T3 FAILED → DEGRADED
T3 dead + T1 wrong	Two sensors disagree, no majority	Heater ON at a real 40 °C → plants cooked	SAFE STATE (heater OFF, fan ON, alarm ON)
T1 stuck	Frozen value	No reaction to the real temperature	TMR voter masks T1
Sensor reports 115 °C	Impossible value	Wrong control action	`ReadingValidator` rejects it

Fault (cause)	Error (wrong state)	Failure if not handled	Prevented by
S1 dead	No soil value	Pump stays ON → flooding	Pump OFF (fail-safe)

Presentation

Run **F1** (or menu `1`). Show `FAULT injected: T2 -> WRONG_VALUE`, the status line `MASKED`, and the summary. Point at `Sensor.sampleOnce()`.

---

3. Role 2 – Fault Tolerance Engineer

> Assignment: TMR voter with ±1 °C tolerance and statuses OK / MASKED / DEGRADED / FAILED. Watchdog with heartbeat (F2) and the safe state with exception handling (F3).

Backend files

File	What to code / explain	Key parts
`fault/TmrVoter.java`	Inexact majority voting (±1.0 °C) and graceful degradation 3 → 2 → safe	`vote()`: 3 agree → OK (median); 2 agree → MASKED (outlier named); only 2 left and they agree → DEGRADED; otherwise → FAILED
`fault/VoteStatus.java`, `fault/VoteResult.java`	Vote status and result	`trustworthy()`
`fault/Watchdog.java`	Periodic task every 1 s. A sensor with no heartbeat for 1.5 s is marked FAILED, and RECOVERED when readings return. Measures the RT3 detection time (≤ 2.5 s).	`check()`, `isHealthy()`, `TIMEOUT_MS = 1500`, RT3 log line
`fault/SafeStateManager.java`	Safe state (heater OFF, fan ON, alarm ON). Leaves it after 2 good votes. Escalates to CRITICAL every 2 s after 30 s.	`enter()`, `goodVote()`, `tick()`, `applyForShutdown()`
`fault/ReadingValidator.java`	Rejects impossible readings (NaN, < −40 / > 80 °C, < 0 / > 100 %)	`validate()`
`api/GlobalExceptionHandler.java`	Every bad request → clear 400 / 404 / 405 / 415 / 500 answer and a log line; the system never crashes	one `@ExceptionHandler` per error type, `reject()`
`api/dto/EnvironmentRequest.java`, `FaultRequest.java`, `HeatWaveRequest.java`	Input validation: required fields and ranges	`@NotNull`, `@DecimalMin`, `@DecimalMax`
`kernel/UnknownSensorException.java`	Unknown sensor id (e.g. `T9`) → 404	–

Shared with Role 4: the `try/catch` in `Controller.run()` is the place where an exception becomes the safe state (concept 5). Role 4 owns the file; Role 2 explains this part in the report and presentation.

Postman folders

Folder	What it shows
F2-DeadSensor	T3 DEAD → watchdog: T3 FAILED within 2.5 s (RT3), voter DEGRADED, recovery after repair
F3-SafeState	T3 dead + T1 wrong → no agreement → SAFE STATE → still safe while the fault lasts → leaves it automatically after repair
X1-InvalidInputs	12 wrong inputs (out of range, text instead of a number, broken JSON, unknown sensor, wrong URL / method) → all rejected, system unchanged

Report section

Redundancy, voting, graceful degradation, watchdog, safe state, forward recovery.

Trustworthy sensors	Agreement (±1 °C)	Vote status	System reaction
3	all 3 agree	OK	Normal control
3	2 agree, 1 outlier	MASKED	Normal control, outlier ignored
2	they agree	DEGRADED	Normal control with 2 sensors
2	they disagree	FAILED	SAFE STATE
0–1	–	FAILED	SAFE STATE

Presentation

Run **F2** and **F3**. Show `WATCHDOG T3 FAILED`, `[RT3 HARD] ... MET`, `ENTER SAFE STATE` and `EXIT SAFE STATE`. Point at `TmrVoter.vote()` and `Watchdog.check()`.

---

4. Role 3 – Concurrency Engineer

> Assignment: Bounded buffer as a Java monitor (wait in a while loop, notifyAll). Correct use of synchronized, volatile and AtomicInteger for all shared data. Race-condition demo with two fixes (F4).

Backend files

File	What to code / explain	Key parts
`buffer/BoundedBuffer.java`	Own circular buffer (size 10) written as a monitor – **no BlockingQueue**	`put()`: `while (full) wait(remaining)` – waits at most 50 ms so a sensor never misses its period; `take()`: `while (empty) wait()`; `notifyAll()` after each change
`buffer/ReadingConsumer.java`	Consumer thread: take → heartbeat → validate → store → Observe & React	`run()` loop ends cleanly on `interrupt()`, `process()`
`buffer/LatestReadings.java`	Shared latest readings and heartbeat times	all methods `synchronized`; `freshValue()`
`demo/RaceConditionDemo.java`	F4: two threads × 100 000 `count++`	`UnsafeCounter` (loses updates), `SynchronizedCounter` (fix 1), `AtomicInteger` (fix 2), `volatile` start gate

Role 3 is also responsible for checking that **every** shared variable in the project is protected correctly (the table below). Example places in other roles' files: `Sensor.fault` (volatile), `Actuator.on` (volatile), `SafeStateManager.active` (volatile), `TimingMonitor.deadlineMisses` (AtomicLong), `Controller.injectException` (AtomicBoolean).

Postman folder

Folder	What it shows
F4-RaceCondition	Unsafe counter < 200 000 (e.g. 130 173, lost 69 827), synchronized = 200 000, AtomicInteger = 200 000

Report section – shared-data table

Shared data	Written by	Read by	Protection
Greenhouse values (`GreenhouseSimulator`)	REST/menu thread, plant task	5 sensor threads, REST	`synchronized` methods
Reading buffer (`BoundedBuffer`)	5 sensor threads	consumer thread	monitor: `synchronized` + `wait` / `notifyAll`
Latest readings + heartbeats (`LatestReadings`)	consumer thread	controller, watchdog, REST	`synchronized` methods
Sensor fault setting (`Sensor.fault`)	REST/menu thread	sensor thread	`volatile` reference to an immutable object
Actuator states (`Actuator.on`)	controller, safe state, consumer (fast path)	display, REST, plant task	`volatile` + `synchronized set()`
Safe-state flag (`SafeStateManager.active`)	controller	display, REST, consumer	`volatile` + `synchronized` methods

Shared data	Written by	Read by	Protection
Watchdog health map	watchdog	controller, display, REST	`synchronized` methods
Controller results (`lastVote` , ...)	controller, consumer	display, REST	`volatile`
Exception-injection flag	REST thread	controller	`AtomicBoolean.getAndSet` (one-shot)
Timing statistics	all periodic tasks	timing report, REST	`synchronized` block + `AtomicLong`
Scenario events (`EventRecorder`)	all threads that log	REST (summary)	`synchronized` methods
Shutdown flag	REST / menu / Spring shutdown	kernel	`AtomicBoolean.compareAndSet` (runs once)

Presentation

Run **F4** (or menu `4`). Show the three counter values. Point at `BoundedBuffer.put()` / `take()` and explain why it is `while` and not `if` .

---

5. Role 4 – Control Engineer

> Assignment: Controller logic – heater, fan, pump with on/off limits; alarm (F5). Actuator class that logs every change with a timestamp.

Backend files

File	What to code / explain	Key parts
`control/Controller.java`	Periodic control cycle every 1 s (Environmental Control) and the Observe & React fast path	`run()` (try/catch → safe state), `controlCycle()`, `controlClimate()` (heater < 22 / > 24 °C with hysteresis, fan > 28 °C or > 85 %, alarm > 35 °C), `controlPump()` (< 35 % / > 50 %), `onTemperatureReading()` (fast path), `reportRt2()`
`control/Actuator.java`	One actuator. Every change is logged with a ms timestamp and the reason.	`set(newState, reason)`
`control/Actuators.java`	Heater, fan, pump and alarm together	`allOff()`, `summary()`

Postman folders

Folder	What it shows
01-Normal-Operation	Heater, fan and pump rules and the heater hysteresis (stays ON between 22 and 24 °C)
F5-HeatWave	+14 °C → alarm + fan ON within 1 s (RT2), e.g. 37 ms
X3-HumiditySoilFailure	S1 dead → pump OFF (no flooding); H1 dead → fan ON (precaution); warnings repeat

Report section – stimulus–response table

Stimulus	Response	Deadline
Temperature < 22 °C	Heater ON	1 s control cycle (RT4, firm)
Temperature > 24 °C	Heater OFF	RT4
Temperature > 28 °C or humidity > 85 %	Fan ON	RT4
Soil < 35 % / > 50 %	Pump ON / OFF	RT4
Temperature > 35 °C	Alarm ON + fan ON (Observe & React)	**1 s (RT2, hard)**
No trustworthy temperature / exception	Safe state: heater OFF, fan ON, alarm ON	next control cycle
Humidity sensor failed	Fan ON (precaution) + repeated warning	RT4
Soil sensor failed	Pump OFF + repeated warning	RT4

Why the fast path exists: without it, the worst case is 500 ms (sensor period) + 1000 ms (control period) ≈ 1.5 s. That would miss the 1 s hard deadline of RT2.

Presentation

Run **F5** (or menu `5`). Show the `Observe & React` line, `ALARM -> ON` and `[RT2 HARD] alarm ON 37 ms ... MET`. Point at `Controller.onTemperatureReading()`.

---

6. Role 5 – Timing & Integration Lead

> Assignment: Main class – starts all periodic tasks with ScheduledExecutorService and shuts everything down cleanly. Status display, fault menu, timing measurement and the timing analysis table. README and run script; group coordinator (meetings, Git, final ZIP).

Backend files

File	What to code / explain	Key parts
`GreenhouseApplication.java`	Main class	`main()`
`kernel/RealTimeKernel.java`	Builds all parts. Starts every periodic task with `scheduleAtFixedRate`, plus the consumer and menu threads. Clean shutdown. Operations used by REST and the menu.	`start()`, `stop()` (`shutdown` → `awaitTermination` → `interrupt` → `join` → final timing report), `requestShutdown()`, `reset()`, `status()`, `scenarioStart()` / `scenarioEnd()`
`monitor/TimingMonitor.java`	`System.nanoTime()` per task, max C, U = C / T, total U vs RM bound 0.757, deadline misses	`record()`, `snapshot()`, `printReport()`, `RM_BOUND`
`monitor/StatusDisplay.java`	Status line every 2 s (RT5, soft)	`printNow()`
`monitor/FaultMenu.java`	Console menu F1–F5, repair, reset, status, timing, summary, exit	`handle()`
`monitor/Log.java`	ms timestamps, CLI + `logs/<date>/greenhouse_<time>.log` + `logs/<date>/events_<time>.log`, markers `>>> !!! +++`, banners, summaries	`write()`, `banner()`, `summary()`
`monitor/EventRecorder.java`	Remembers the important events of the current scenario (repeated warnings collapsed)	`record()`, `start()`
`monitor/InputDescriber.java`	Plain-words text for the `NEW INPUT RECEIVED` box: what an input means and what is expected	`faultMeaning()`, `faultExpected()`, `environmentExpected()`
`api/GreenhouseRestController.java`	REST endpoints used by Postman (each calls a kernel operation)	one method per endpoint
`api/dto/ScenarioRequest.java`	Body for the scenario banner	–

Postman and scripts

File / folder	What it is
00-Common-Setup	System running → reset → normal values → vote OK
02-Repair-All	Repairs every sensor, back to normal
03-Timing-And-Shutdown	Timing table (asserts total U ≤ 0.757) + clean shutdown
X4-MixedChaos	All roles' features mixed: valid + invalid inputs, faults, exception, heat wave → never crashes (integration test)
`postman/build-collection.js`	Generates the collection and both environments: `node postman/build-collection.js`

File / folder	What it is
`postman/greenhouse*.postman_environment.json`	Fast (`stepDelay = 0`) and presentation (`stepDelay = 3000`) environments
`pom.xml`, `mvnw`, `mvnw.cmd`, `.mvn/`, `application.properties`	Build and settings
`run.bat` / `run.sh`	One-command start (finds Java 21, builds, runs)
`run-demo.bat`, `test-scenario.bat`, `test-all.bat`	Automatic runs with Newman
`README.md`, `CONTRIBUTIONS.md`	Documentation

Report section

- Section 4 timing table: take the `TIMING REPORT (FINAL)` from `logs/<date>/greenhouse_<time>.log`.
- RM test: total U vs n(2^(1/n) − 1) = 0.757 for n = 4.
- Why normal Java cannot guarantee deadlines:
  - **Garbage collection** pauses all threads.
  - **The OS scheduler** decides when threads run, and Java priorities are only hints.
  - **JIT warm-up** makes the first runs slower.

  So max C is only an estimate of the WCET.

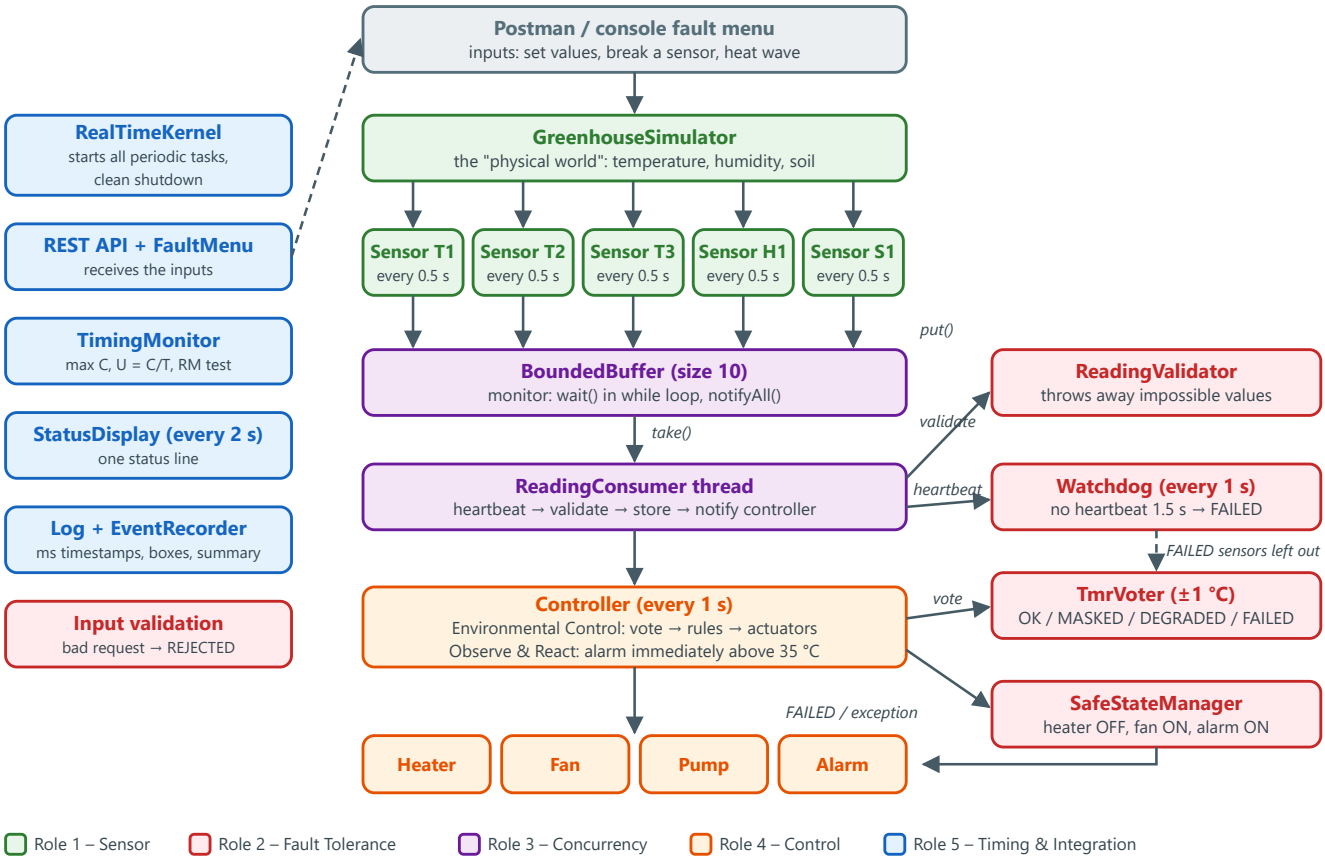
Presentation and coordination

- Start the system with `run.bat`, run the full collection (or `run-demo.bat slow`) and show the final timing table.
- Coordinate meetings and the Git repository. Prepare the final ZIP: the project folder **without** `target/`, `logs/` and `reports/`.

---

7. Where the roles' code meets (agree on these first)

System architecture – which role owns which part



Arrows show the data path of one reading: input → simulator → sensor → buffer → consumer → controller → actuators.

Hand-over point	From → To	Method / data
Sensor reads the greenhouse	Role 1 → Role 1	`GreenhouseSimulator.read(type)`
Sensor puts a reading into the buffer	Role 1 → Role 3	`BoundedBuffer.put(Reading, timeoutMs)`
Consumer validates a reading	Role 3 → Role 2	`ReadingValidator.validate(Reading)`
Consumer records the heartbeat	Role 3 → Role 2	`LatestReadings.heartbeat()` → read by `Watchdog.check()`
Consumer triggers Observe & React	Role 3 → Role 4	`Controller.onTemperatureReading()`
Controller asks for trustworthy data	Role 4 → Roles 2, 3	`Watchdog.isHealthy(id)`, `LatestReadings.freshValue()`
Controller votes	Role 4 → Role 2	`TmrVoter.vote(Map<id, value>)` → `VoteResult`
Controller enters / leaves the safe state	Role 4 → Role 2	`SafeStateManager.enter()`, `goodVote()`, `tick()`
Safe state switches actuators	Role 2 → Role 4	`Actuator.set(state, reason)`
Every task reports its execution time	Roles 1–4 → Role 5	`TimingMonitor.record(task, instance, start, end)`
Every event is logged	all → Role 5	`Log.info / warn / error(tag, message)`
REST / menu trigger a fault	Role 5 → Role 1	`Sensor.injectFault(mode, value)`, `repair()`

Hand-over point	From → To	Method / data
Kernel starts / stops all tasks	Role 5 → all	`scheduleAtFixedRate(...)`, `shutdown()`, `interrupt()`

8. Suggested Git workflow

1. **Repository (Role 5):** creates it on GitHub with the current project on `main`.
2. **Branches:** each member works on their own branch – `role1-sensors`, `role2-fault-tolerance`, `role3-concurrency`, `role4-control`, `role5-integration`.
3. **Edit only your own files.** If you need a change in another role's file, ask its owner, or agree on it in the hand-over table above.
4. **Merge (Role 5):** merges each branch into `main` and runs `run-demo.bat` before every merge. All Postman checks must pass.
5. **Concept tags:** each member checks that their `// CONCEPT n:` tags and `Author:` headers are correct before the final ZIP.

---

9. Lecture concepts → owner

#	Concept	Week	Owner	Where
1	Real-time system, periodic tasks, hard/firm/soft deadlines, sensors & actuators	1	Roles 1, 4	`Sensor.run()`, `Controller`, `Actuator`, `TimingMonitor.Task` (RT1–RT5)
2	Fault → error → failure; fail-safe state	1, 2	Roles 1, 2	`FaultMode`, `Sensor.sampleOnce()`, `ReadingValidator`, `SafeStateManager.enter()`
3	TMR with inexact voting; graceful degradation	3, 4	Role 2	`TmrVoter.vote()`
4	Watchdog / heartbeat	4, 5	Role 2	`Watchdog.check()`, heartbeat recorded in `ReadingConsumer.process()`
5	Exception handling to a safe state (forward recovery)	5	Role 2 (code in Role 4's `Controller.run()`)	`Controller.run()` try/catch → `SafeStateManager.enter()`, `GlobalExceptionHandler`
6	Threads, Runnable, ScheduledExecutorService, interrupt, clean shutdown	6	Roles 1, 5	`Sensor implements Runnable`, `RealTimeKernel.start()` / `stop()`, `ReadingConsumer.run()`
7	volatile and atomic; race on count++	7, 8	Role 3	`RaceConditionDemo`, volatile/atomic fields listed in the shared-data table
8	synchronized methods / blocks	9	Role 3	`LatestReadings`, `GreenhouseSimulator`, `TimingMonitor.record()`, `SynchronizedCounter`
9	Monitor bounded buffer	11	Role 3	`BoundedBuffer.put()` / `take()`
10	Environmental Control and Observe & React	13	Role 4	`Controller.controlCycle()`, `Controller.onTemperatureReading()`
11	Execution-time measurement and RM test	14	Role 5	`TimingMonitor.record()` / `printReport()`